

# Process Report: Waterfall Reflection

Software Engineering - SVNC.00.308

Project: StayEase Hotel Reservation System

Type: Hotel Reservation

Team: Cat Bacillus (Kyrylo Pryiomyshev, Danylo Chernov, Daniil Ustuhov)

Date: February 20, 2026

Version: 1.0

## 1. Process Summary

Our team developed documentation for the StayEase Hotel Reservation System, a web-based platform for Grand Vista Hotel to manage room bookings, guest check-ins, and occupancy tracking. Over two lecture sessions, we followed the Waterfall methodology sequentially: conducting a stakeholder interview, documenting requirements, creating a project plan with Gantt chart, designing the system architecture and data model, and developing a test plan. We completed Phase Gates 1 through 3, obtaining approval at each stage. Key deliverables include the Requirements Specification (12 functional and 5 non-functional requirements), Design Documentation (three-tier architecture with React, Express, PostgreSQL), Test Plan (15 test cases with traceability matrix), and the Project Plan.

## 2. What Worked Well

The sequential structure gave our team a clear roadmap. Completing requirements before design meant we thoroughly understood the hotel's needs, which directly influenced architectural decisions. For instance, knowing that double-booking prevention was high-priority led us to choose PostgreSQL with ACID transactions over a NoSQL alternative. Phase gate reviews proved valuable as quality checkpoints. During Phase Gate 1, we caught a missing check-in/check-out feature that the hotel manager had mentioned but we had not formally documented. Fixing this before design was far cheaper than discovering it during implementation. The comprehensive documentation also ensured all three team members shared a consistent understanding of the system, even while working on different components.

## 3. Challenges and Limitations

The biggest challenge was the rigidity of completing each phase before proceeding. During design, we discovered edge cases in reservation modification (what happens when a guest extends their stay into dates already booked by someone else) that we had not anticipated in requirements. Revising the requirements document at that stage felt inefficient. A prototype-first approach might have surfaced this gap sooner. The documentation overhead was also substantial for a three-hour exercise. While detailed specs would be justified in a real multi-month project, for our compressed scope we spent more time documenting the system than we would have spent building a basic working version.

## 4. Comparison to Alternatives

An Agile approach would have let us prioritize the three most critical features (room search, booking, double-booking prevention), build a working prototype in the first lecture, and gather stakeholder feedback immediately. This would have provided faster validation. However, we would have lost the upfront architectural planning that Waterfall encouraged. For a reservation system where data integrity is critical, that early design thinking was genuinely valuable and might have prevented costly refactoring later.

## 5. When to Use Waterfall

Waterfall fits projects with stable, well-understood requirements where late-stage changes are costly. A production hotel reservation system integrating with external booking platforms or

handling payments would benefit from thorough documentation and formal approvals. Regulated industries (healthcare, finance) similarly need Waterfall's audit trails. Large distributed teams also benefit from detailed specifications as contracts between groups. However, for startups exploring new ideas or projects with uncertain requirements, Agile provides the flexibility that Waterfall cannot.

## 6. Lessons Learned

Requirements gathering is harder than it appears: even with a structured interview, we missed edge cases that only surfaced during design. Documentation has real value as a reference artifact, not just busywork. Phase gates catch errors at the cheapest point to fix them. Most importantly, there is no universally best methodology. Waterfall offers predictability and thoroughness at the cost of flexibility; Agile offers adaptability at the cost of upfront planning. The right choice depends on project context.

## 7. Conclusion

This exercise demonstrated that Waterfall is effective when applied in the right context. We experienced both its strengths (structured teamwork, early error detection, comprehensive planning) and its limitations (rigidity, documentation overhead). The key lesson is that successful software engineering means choosing the right process for the project's constraints, not blindly following any single methodology.

## 8. Team Contributions

| Team Member        | Role                       | Contributions                                                                            |
|--------------------|----------------------------|------------------------------------------------------------------------------------------|
| Kyrylo Pryiomyshev | PM / Backend Dev           | Led requirements gathering, created Gantt chart, designed backend API endpoints          |
| Danylo Chernov     | Design Lead / Frontend Dev | Designed system architecture, UI mockups, documented tech stack and trade-offs           |
| Daniil Ustiuhov    | QA Lead / DB Developer     | Designed database schema, wrote test plan with 15 test cases, managed phase gate reviews |

**Team Dynamics:** We collaborated throughout both sessions, with clear role assignments preventing duplication. Regular phase gate check-ins ensured alignment before proceeding.

## Sign-Off

**Prepared By:** Cat Bacillus (Kyrylo Pryiomyshev, Danylo Chernov, Daniil Ustiuhov)

**Date:** February 20, 2026